Team Name : CodeRAG

Team Leader Name : Konnuru Yashwanth

Problem Statement :

Recruiters scan hundreds of profiles and still miss the right person, because keyword filters can't see what actually matters. The task is to build an AI system that reads a job description, understands the full picture — career history, skills, behavioral signals, platform activity — and returns a top-100 shortlist a recruiter can trust, drawn from a 100,000-candidate pool, running on CPU with no network.

01 / Architecture

Solution Overview & Key Differentiators

THE PROPOSED SOLUTION

Offline, Recruiter-Grade Ranking Engine

A robust system computing merit-based shortlists locally on CPU, with zero network dependencies.

7-Factor Fit Scoring: Computes seven explainable candidate signals mapping direct fit against JD constraints.

Availability & Penalty Adjustments: Modifies scores using real-world platform activity while penalizing explicit JD disqualifiers.

Shortlist with Justifications: Outputs a top-100 CSV featuring recruiter-grade, fact-based logic for every decision.

01 Roles Over Keywords

Prioritizes real career evidence and core titles over keyword-stuffed lists. Correctly ranks down candidates with mismatched actual experience.

02 Trust-Weighted Skills

Defeats keyword manipulation. Unendorsed skills with zero active usage months are discounted, focusing only on verified expertise.

03 Honeypot Detection

Algorithmic safety checks automatically isolate internally impossible profiles and push them to the bottom without hardcoding exceptions.

04 Availability-Aware

De-prioritizes unresponsive, inactive candidates. Ensures the shortlist contains highly hireable, active professionals, not dead leads.

02 / Evaluation

JD Understanding & Candidate Evaluation

Key Requirements Extracted from JD

- **Experience Focus:** 5–9 years (ideal 6–8) in applied ML/AI at product companies, not pure services or research.
- **Technical Stack:** Embeddings retrieval, vector databases, or hybrid search (FAISS, Pinecone, Elasticsearch, Qdrant).
- **System Experience:** Shipped ranking, search, or recommendation systems to real users with rigorous evaluation (NDCG, A/B testing).
- **Domain & Location:** NLP/IR background (no pure CV/robotics). Based in or willing to relocate to an Indian metro.
- **Strict Disqualifiers:** Consulting/research-only, title-chasers, and "architect/manager" candidates who no longer code.

Decisive Signals & Beyond Keywords

1. Career History Over Buzzwords

We prioritize actual job titles and evidence-backed career descriptions over keyword-stuffed profiles. A plain-language engineer who actually built a system surfaces, while keyword-spammers are filtered out.

2. Verifiable Skill Trustworthiness

Unendorsed skills with zero active usage months are discounted. Relevance is determined by skill endorsements, total years used, and Redrob assessment scores.

3. Behavioral Availability

Rather than chasing dead leads, the system boosts active talent using real platform metrics: recruiter response rates, recent activity, and "open-to-work" status.

03 / Architecture

Ranking Methodology & Mathematical Framework

Algorithmic Pipeline & Models

Stream-Based Execution: The 100,000-candidate file is streamed once. We compute seven structured signals, apply red-flag penalties, and run honeypot checks with zero network dependencies.

Hybrid Semantic Engine: A second pass builds TF-IDF vectors (via scikit-learn) of every profile to measure cosine similarity against a JD-derived query. This is fast, deterministic, and fits our strict CPU budget.

Multi-Signal Formula: Combines weights multiplicatively to prevent dead leads:
Score = (Weighted Merit) × Red-Flag Penalty × Behavioral Modifier

Deterministic Selection: Candidate ties are broken by candidate_id to output a stable top-100 list.

Component Weight Distribution (Weighted Merit)

04 / Validation

Explainability & Data Validation

1. Decision Justification

How are ranking decisions explained?

Every row receives a **one-to-two sentence justification** built directly from structured profile facts: current title, years of experience, named real skills, and signal values.

Concerns like short tenure or long notice periods are stated honestly, adjusting the tone dynamically to match the candidate's rank.

2. Fact-Based Reasoning

How do you prevent hallucinations?

Reasoning is assembled strictly from **verifiable, structured fields** rather than free-text generated by a large language model.

A skill, employer, or experience level physically cannot appear in the explanation unless it exists in the active profile—eliminating hallucinations by construction.

3. Honeypot & Quality Checks

How are bad profiles handled?

Internal-consistency checks catch impossible timelines (e.g., role duration exceeding start/end dates, or skill usage longer than entire career).

This system flagged and collapsed **43 impossible profiles** to the bottom, resulting in zero honeypots reaching the final top 100.

05 / Execution

End-to-End Workflow

01

JD → Role Spec

The job description is parsed once, offline, into structured requirements, traps, and disqualifiers, committed as `role_spec.yaml` so ranking needs no network.

02

Stream 100k

Candidates JSONL is streamed once. Per candidate, seven structured signals, red-flag penalties, behavioral modifiers, and honeypots are computed.

03

Semantic Pass

TF-IDF builds profile vectors and scores cosine similarity directly against the JD query, applying a MinMax normalization across all records.

04

Combine & Rank

Multiplies merit × penalties × availability. Honeypots are collapsed to the bottom. Sorted and tie-broken by ID to output the stable top 100.

05

Write CSV

Generates a validator-compliant `submission.csv` featuring per-row reasons based strictly on structured profile facts.

SINGLE COMMAND EXECUTION

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

(~1 minute, CPU-only, offline)

06 / Architecture

System Architecture

1. Core Package

Auditable & Defensible

The system is a small, modular Python package where each module owns one clean responsibility.

pipeline.py
Orchestrates the single streaming pass and the semantic pass.

Zero-Latency Ranking
No network, GPU, or hosted-LLM call occurs at ranking time.

2. Data & Features

Stream & Signal Extraction

data.py / text.py
Streams the jsonl/gz pool and builds each candidate's text profile.

features.py
Extracts seven structured signals, including trust-weighted skills.

semantic.py
Runs TF-IDF similarity with an optional dense-embedding backend.

3. Behavioral & Reasoning

Filters & Explanations

behavioral.py
Applies the availability multiplier derived from Redrob signals.

honeypot.py
Runs internal-consistency checks that collapse impossible profiles.

reasoning.py
Generates fact-grounded, transparent per-row explanations.

External Configuration
`config.yaml` holds weights and `role_spec.yaml` holds JD understanding.

07 / Evaluation

Results & Performance

1. Ranking Quality & Precision

- **High-Target Precision:** The top 100 shortlisted from the 100,000-candidate pool is composed almost entirely of the target profile (e.g., 14 AI, 12 ML, 10 Applied-ML, and 8 Recommendation-Systems engineers).
- **Experience Target:** 92 of the 100 candidates fall strictly inside the requested 5–9 year band, with a mean of **6.6 years**.
- **Zero False Positives:** Active honeypot filtering and semantic reasoning ensured **0 keyword-stuffer roles** (no Marketing, HR, or Accountants) made the shortlist.

COGNITIVE MATCHING

2. Constraints & Efficiency

- **Speed & Runtime:** The entire 100k pipeline executes in **~1 minute** on a standard laptop CPU, comfortably below the 5-minute budget.
- **Hardware Bounds:** Operates on **<16 GB RAM** with no GPU reliance, and requires zero network or API integrations during ranking.
- **Frictionless Verification:** Output strictly adheres to the official formatting and cleanly passes the **validate_submission.py** checker.
- **Self-Contained TF-IDF:** Selected to ensure offline, reproducible execution. Highly scalable dense-embeddings are supported but completely optional.

05 / Technology

Technologies Used

Core Stack & Selection Rationale

Python 3.10

Strong, readable production code, which the JD treats as a core requirement.

scikit-learn (TF-IDF)

Fast, deterministic semantic similarity with no model download, keeping ranking offline.

NumPy

Vectorized scoring and sorting across 100k candidates in seconds.

PyYAML

Human-readable configuration and the committed JD understanding (role_spec.yaml).

Streamlit & Git

Free hosted sandbox for browser testing and a versioned, reproducible repository.

The Lean Stack Strategy

Our architectural stack is a deliberate **latency-quality decision** tailored for production-scale constraints.

A system that calls a hosted LLM for each of 100k+ candidates **cannot meet a 5-minute CPU budget** in production.

Instead, we engineered a **small, fast, and fully-offline ranker** that delivers immediate results on consumer hardware without sacrificing analytical rigor.

Submission Assets

Github video etc

08 / Deliverables

Submission Assets

1. GitHub Repository

Clean production-grade Python code with a complete README for single-command replication.

`github.com/YashhCanCode/RedRob-Ranker`

2. Live Sandbox Demo

An interactive Streamlit web application to run and evaluate the offline ranker on candidate samples.

`redrob-rank.streamlit.app`

3. Ranked Output & Approach

submission.csv: The finalized top-100 candidates with structured, fact-based justifications.

SUBMITTED BY

Team codeRAG

"Make hiring smarter."

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU